

SkyGuard AI

The AI That Asks: "Is the Weather Lying, or Is the Sensor?"

PS Code: SIH26073 | Track: Software | Theme: Disaster Management

Sponsoring Ministry: Ministry of Earth Sciences (MoES)

Final Integrated Team Solution — Best-of-Both Blueprint + X-Factor Edition

This document merges your Team Technical Blueprint and the Complete Project Blueprint into one final build plan, explained in plain language, with the strongest features from both kept and one brand-new X-Factor added to make the demo genuinely unforgettable for judges.

1. The Idea in One Sentence

SkyGuard AI looks at a weird weather reading and figures out — like a detective — whether the weather is actually doing something unusual, or whether a sensor has gone faulty. Then it explains its reasoning in plain words, suggests a safer number to use instead, keeps score of how healthy every sensor is over time, and tells the maintenance team exactly which sensor to fix first.

Most competing teams will build something that says *"this number looks wrong."* SkyGuard says *"this number looks wrong, here's why, here's what it probably should be, here's how sick this sensor has been getting for the last 3 weeks, and here's what would have happened downstream if we hadn't caught it."* That last part is our X-Factor — more on that in Section 9.

2. The Problem, Explained Simply

Weather stations spread across India constantly report temperature, air pressure, and humidity — completely unmanned, 24/7. Like any machine left outside in the sun, rain, and dust for years, their sensors slowly go bad: a wire corrodes, a chip drifts out of calibration, a connection drops. When that happens, the station starts sending wrong numbers — but it doesn't know it's wrong, and neither does anyone else, until a forecast or a flood warning based on that bad number turns out to be wrong too.

The tricky part is that **a weird number isn't always a broken sensor**. A sudden temperature spike could be a real heatwave. A pressure crash could be a real approaching storm. The system has to be smart enough to tell these two situations apart — and today's basic "alert if the number crosses a fixed limit" systems simply can't.

3. Who This Actually Helps

Who	What They Deal With Today	How SkyGuard Helps Them
MoES / IMD Data Analyst	Manually eyeballing hundreds of station feeds, using rigid threshold rules	Gets a ranked list of real problems with plain-English reasons, not raw numbers to sift through
Disaster Management Teams (NDMA, State DM Cells)	Risk of a flood/cyclone/heatwave alert being wrong because the input data was wrong	Gets a trust score on every reading feeding their models — knows exactly how much to trust it
Field Maintenance Engineers	Fixed-schedule visits regardless of which sensor is actually failing	Gets a prioritized repair list: "fix this one first, it's been declining for 3 weeks"
Farmers (via Agri-Advisory)	Advisories based on whatever number the station reports, right or wrong	Indirectly protected — bad readings get caught before they turn into bad advice
Pilots / Aviation Met Officers	Trust briefings without knowing if the underlying sensor is reliable	Gets confidence-scored data instead of blind trust in a single number

4. Empathy Canvas

Meet Priya, a Data Quality Analyst at a State IMD Regional Centre.

Says	Thinks
------	--------

"I've got 200 station feeds and a spreadsheet of fixed thresholds — I can't tell which alerts are real."	"Half these alerts are probably nothing. But if I ignore the wrong one, someone gets a bad flood warning."
Does	Feels
Manually checks flagged outliers, waits for a field visit report or a public complaint to confirm a fault.	Alert fatigue and quiet anxiety — worried about missing the one reading that actually mattered.

Before → **After:** "I find out a reading was garbage after a bad forecast already went out" → "I get a ranked alert within seconds: 'AWS-104 temperature — 97% likely sensor spike, here's the evidence, here's the safe value to use instead, and here's how this sensor's health has been trending for the last month.'"

5. What We Are Building — The Six Core Features

These are the strongest ideas pulled from both of your team blueprints, merged into one coherent product (not six separate gadgets — one connected story):

#	Feature	In Plain Words
1	Smart Anomaly Detection	Combines simple rule-checks, statistics, and machine learning together — because no single method catches every type of sensor problem (sudden spikes, slow drift, frozen values, missing data).
2	Weather-vs-Sensor Engine (the brain)	For every weird reading, checks: did nearby stations see the same thing? Did the other sensors (pressure, humidity) move in a way that agrees? Only then decides real weather vs. broken sensor.
3	AWS Trust Score (0-100)	A single, simple number telling anyone — analyst or automated system — how much to trust a given reading right now, instead of a confusing pile of raw stats.
4	Sensor Digital Twin	A living health profile for every physical sensor: how reliable has it been, is it slowly getting worse, when did it last misbehave — like a health record for each sensor.
5	Explainable Correction (Human-Approved)	Never silently changes the real data. It proposes a safer estimated value with its reasoning shown, and a human clicks Accept / Reject / Review — the raw reading is always preserved.
6	Predictive Maintenance Queue	Turns all the health data into a simple ranked to-do list for field engineers: which sensor to inspect first, and why.

6. How It All Connects (Architecture)

```
AWS Sensor Data (real / historical / simulated)
v
Data Ingestion + Validation (catch missing values, bad timestamps, impossible numbers)
v
Feature Engineering (rolling averages, rate-of-change, time-of-day/season patterns)
v
+-----+-----+
| Statistical | ML Detectors |
| Rules / EWMA | Isolation Forest |
| | + Autoencoder |
+-----+-----+
v
Consistency Engine (Temporal + Cross-Sensor + Cross-Station + Persistence checks)
v
WEATHER-vs-SENSOR DECISION ENGINE <-- the core brain
v
Fault Classifier (spike / drift / frozen / dropout / multivariate)
v
Trust Score + Explainability + Suggested Correction (human-approved)
v
Sensor Digital Twin ---> Predictive Maintenance Queue
v
Live Dashboard (Command Center, Map, Investigation View, Maintenance Queue)
```

7. Dataset Strategy — What Data We Actually Need

Good news: nothing here requires private or classified data. The two-layer approach both your blueprints agree on is the right one:

- **Layer 1 — Real historical data** to learn what "normal" looks like: public IMD/NOAA weather station records, at least 10-20 stations across 3-6 months of hourly Temperature/Pressure/Humidity data for a solid demo (1 station is enough for a bare MVP).
- **Layer 2 — Synthetic fault injection** to create labeled "broken sensor" examples, since no public dataset comes with real fault labels: inject spikes, slow drift, frozen (flatline) values, dropouts, abrupt jumps, and cross-sensor inconsistencies into the clean data, with exact ground-truth labels for each.
- Keep the model built around only the three official inputs — **Temperature, Pressure, Humidity** — plus features calculated from them (rolling average, rate of change, time-of-day). Don't make the core model depend on extra variables (wind, rainfall, etc.) that the PS didn't ask for.

Volume guide: 500-2,000 synthetic fault events spread across all fault types is enough to train and fairly test the detector; hold back some stations/time periods entirely for testing so the model proves it generalizes, not just memorizes.

8. Build Plan — Phase by Phase

Phase	What Happens
0. Scope Freeze	Lock in T/P/H as the only core inputs, agree the MVP boundary, set up GitHub branches and task board
1. Data Audit + Pipeline	Pull the real dataset, check its columns/timestamps/stations, build clean ingestion + validation
2. Fault Injection Lab	Build reproducible spike/drift/frozen/dropout/multivariate fault generators with ground-truth labels
3. Feature Engineering	Build rolling stats, rate-of-change, and time-of-day/season features
4. Baseline Detectors	Statistical rules first (fast, explainable) — get a working baseline before touching ML
5. ML Layer	Add Isolation Forest, then an Autoencoder trained only on clean data
6. Consistency + Weather-vs-Sensor Engine	The core brain: combine temporal + cross-sensor + cross-station evidence into the real-vs-fault decision
7. Trust Score + Fault Classifier + Explanation	Turn the decision into a 0-100 trust score, a fault-type label, and a plain-English reason
8. Correction + Digital Twin + Maintenance	Suggested safe value (human-approved), sensor health history, ranked maintenance queue
9. Backend + Dashboard	FastAPI + WebSockets backend; React dashboard (Command Center, Map, Investigation, Maintenance)
10. X-Factor Module	Build the Cascade Impact Simulator (Section 9) — this is what makes the demo memorable
11. Evaluation + Demo Engineering	Run the fault-injection benchmark, build 3-4 deterministic demo scenarios
12. Polish + Rehearsal	Docker packaging, stress-test on a clean machine, rehearse the pitch and Q&A;

9. THE X-FACTOR — What Makes Judges Remember You

Both your existing blueprints are already excellent — Trust Score, Digital Twin, Weather-vs-Sensor reasoning, and human-approved correction are genuinely strong, well thought-out features. But almost every strong team at SIH will show *some* version of "we detected the anomaly and explained it." To win, you need one moment in your demo that no other team has — something that makes a judge lean forward. Here it is:

X-Factor: The Cascade Impact Simulator — "Show the disaster we prevented, not just the anomaly we caught."

The idea in plain words: Right now, your system (and every competing team's system) stops at "we caught a bad reading and explained why." That's good, but it's abstract — judges can't feel the stakes. The Cascade Impact Simulator goes one step further: it shows, live on screen, **what would have happened downstream if the bad reading had NOT been caught.**

Concretely: when SkyGuard flags a faulty reading (say, a temperature sensor falsely spiking to 55°C), the system runs that bad value through a simplified version of a real downstream use case — for example, a basic heatwave-alert rule ("if forecast area average exceeds threshold X, issue a heatwave warning") — and shows: **"If this bad reading had gone uncaught, it would have triggered a false heatwave alert for this district."** Or on the flip side, for a genuine extreme value SkyGuard correctly keeps: **"This reading is real — suppressing it as 'anomalous' would have delayed a genuine heatwave warning by approximately 6 hours."**

This turns your demo from *"look at our clever anomaly detector"* into *"watch us prevent a false flood alert / catch a real one in real time"* — which is a completely different, far more memorable experience for a judging panel, and it directly and visibly answers the PS's own "Impact" and "Real-Time Capability" evaluation criteria in one screen.

Why this is genuinely achievable in your timeline (not scope creep):

- You already have the trust score, fault classification, and corrected-value estimate built (Phases 6-8) — the Cascade Simulator is a thin extra layer on top, not a new ML system.
- The "downstream rule" doesn't need to be a real IMD forecasting model — a simple, clearly-labeled toy rule (e.g. a basic heatwave/flood threshold check) is enough to make the point honestly. State this plainly to judges: "this is a simplified illustrative rule, not IMD's actual forecasting model" — honesty here builds credibility rather than costing you points.
- It reuses your existing Fault Injection Lab (already planned) — you just add one more visual layer that runs the injected-fault scenario through the toy downstream rule and shows the before/after outcome.

Secondary X-Factor (if time allows): The AWS Trust Passport

Every reading that passes through SkyGuard gets a small, tamper-evident "trust certificate" — a hash-linked record showing its trust score, evidence, and decision at the moment it was processed (borrowing the audit-log idea already in your Team Blueprint's Repo 1 analysis, but reframed as a memorable, visual "passport stamp" on the dashboard rather than a backend log line). This gives you a strong closing line: *"Every number that reaches a forecaster or a farmer has a passport — you can always see exactly why it was trusted."*

10. Recommended Tech Stack

Layer	Technology	Why
Frontend	React + Vite + Tailwind CSS	Fast, component-based, easy to make look polished quickly
Charts / Map	Recharts or Plotly + Leaflet	Time-series graphs and a live station map
Backend	FastAPI	Python-native, integrates directly with your ML code, fast to build APIs
Real-time	WebSockets	Live/replay event streaming to the dashboard
ML — Classical	scikit-learn (Isolation Forest)	Fast, well-understood unsupervised anomaly detector
ML — Deep Learning	PyTorch (Autoencoder)	Learns "normal" patterns, flags high reconstruction error as anomalous
Data Handling	Pandas + NumPy	Preprocessing and feature engineering
Database	PostgreSQL / Supabase	Stores stations, readings, incidents, health history
Validation	Pydantic	API input/output contracts
Deployment	Docker + Vercel/Railway	Reproducible, judge can run/access it easily

11. Six-Member Team Division

Member	Owns	Delivers
1 — Data/ML	Dataset + preprocessing + fault injection	Clean dataset, labeled synthetic faults, train/test split
2 — ML Models	Isolation Forest + Autoencoder	Trained models, anomaly scores, evaluation report
3 — Reasoning	Consistency engine + Weather-vs-Sensor + fault classifier + Trust Score	The core decision engine and explanations
4 — Backend	FastAPI + database + WebSockets	APIs, schema, live inference service
5 — Frontend	React dashboard	Command Center, map, investigation view, maintenance queue
6 — Integration/Demo	Fault Injection Lab + Cascade Impact Simulator (X-Factor) + deployment	Live demo orchestration, Docker packaging, pitch rehearsal

12. The Winning Demo Script

- 1 Open the Command Center — show a healthy network, all stations green, high trust scores.
- 2 Inject a sudden 55°C temperature spike at one station while neighboring stations stay normal.
- 3 Show SkyGuard flag it in real time, walk through the evidence (temporal + cross-sensor + cross-station).
- 4 Show the classification ("probable sensor spike"), trust score drop, and plain-English explanation.
- 5 **Trigger the Cascade Impact Simulator:** show what a false heatwave alert would have looked like if this reading had gone uncaught — this is your standout moment.
- 6 Show the suggested corrected value, with Accept/Reject/Review controls — raw data untouched.
- 7 Open that sensor's Digital Twin — show its health score declining over recent injected faults, and its new position in the maintenance queue.
- 8 Run a second scenario: a coordinated heat event across several nearby stations — show SkyGuard correctly identifies it as genuine weather, not a sensor fault, and suppresses a false alarm.
- 9 Close on the Network Health view and the one-line pitch: "SkyGuard doesn't just detect a bad number — it tells you whether to trust it, why, what it should have been, and what would have happened if you hadn't caught it."

13. How We'll Measure Success

Metric	Why It Matters
Precision / Recall / F1	Balances catching real faults against not crying wolf too often
False Alarm Rate During Genuine Weather	Proves the Weather-vs-Sensor engine actually works, not just a re-skinned outlier detector
Detection Latency	Proves the "real-time" claim with an actual number, not just a word
Correction Error (MAE/RMSE)	Measures how good the suggested corrected values actually are
Early Degradation Warning Time	Shows the predictive maintenance feature has real lead time, not just a label
Cascade Impact Cases Shown	A qualitative but powerful metric: how many concrete "prevented false alert" scenarios you can demonstrate

14. Why This Beats a Typical Competing Project

A Typical Team's Project	SkyGuard AI
Flags a number as "anomalous"	Investigates the anomaly using multiple independent signals before deciding
Fixed threshold or a single ML model	Hybrid: statistics + ML + cross-sensor + cross-station evidence
Treats every extreme reading as bad data	Explicitly distinguishes real weather events from sensor faults
Shows an anomaly score	Shows a Trust Score, fault type, confidence, and plain-English evidence

A Typical Team's Project	SkyGuard AI
Stops at detection	Continues to correction, sensor health, and maintenance priority
Silently overwrites or deletes bad data	Quarantines it and always preserves the original raw value
Static dashboard over old data	Live/replay streaming demo with real-time alerts
"We caught an anomaly" (abstract)	"Here's the false alert we just prevented" (the X-Factor, concrete and visual)

15. Honest Limitations (Say These Before a Judge Asks)

- **Synthetic fault labels, not real fault logs** — no public dataset provides real, verified sensor-fault events at scale, so evaluation uses synthetically injected faults with known ground truth. State this upfront; it's standard, defensible research practice, not a shortcut.
- **The Cascade Impact Simulator uses a simplified illustrative rule**, not IMD's real forecasting model — say this plainly. The point is to demonstrate the concept of downstream impact, not to replicate a classified/proprietary forecasting system.
- **"Real-time" in the demo means a timed replay of historical/synthetic data**, not a live production AWS feed — this is standard hackathon practice, but be transparent about it.
- **Cross-station correlation needs geographically close reference stations** — sparse-network regions will have weaker correlation confidence than dense ones.
- **Don't overclaim "self-healing"** — corrections are always proposed and human-approved, never silent or automatic, and the raw reading is always preserved.

16. Final Evaluation Against SIH Judging Weights

Judging Criterion (with official weight)	How SkyGuard Delivers
Innovation & Novelty (25%)	Weather-vs-Sensor reasoning + Trust Score + Digital Twin + Cascade Impact Simulator (X-Factor) — a combination not seen together in reviewed reference repos
Detection Accuracy (20%)	Hybrid statistical + ML ensemble, evaluated on a controlled fault-injection benchmark with precision/recall/F1
Real-Time Capability (15%)	Streaming/replay architecture with measured detection latency
Explainability (10%)	Plain-English evidence chain + confidence decomposition, not a black-box score
Scalability (10%)	Station → district → state/network architecture, API-first design
Practical Deployability (10%)	FastAPI + Docker + modular ML pipeline, deployable end-to-end
Visualization / UI (5%)	Command Center, map, investigation view, maintenance queue
Energy Efficiency (5%)	Edge deployment kept as an optional stretch extension, not a core dependency

17. Final One-Line Pitch

"Other systems ask: *'Is this reading anomalous?'* SkyGuard asks: *'Is the weather anomalous, or is the sensor lying — and what would have happened if we hadn't caught it?'* Then it explains, corrects safely, tracks sensor health, and shows the disaster it just prevented."